Chapter 4 練習

作答時標明 pipeline stage、cycle、register 名稱、forwarding source 與 stall/flush 原因。未指定的範例採用教材五階、單發射且沒有 architectural branch delay slot 的簡化 pipeline；第2章的傳統MIPS程式延遲槽規則不直接套用到本章flush模型。

CH04-Q01 單週期臨界路徑

某 MIPS subset 的 single-cycle 元件延遲如下，單位皆為 ps。

元件	延遲
PC clock-to-Q	30
instruction memory	200
register read	120
ALU	180
data memory	250
mux	20
register setup	40

`lw` 會經過 PC、instruction memory、register read、ALU、data memory、memory-to-register mux，再到 register setup；本題簡化 `R-type` path 同時略過 data memory 與 memory-to-register mux，其他列出的延遲不變。

1. 以 `lw` path 計算可接受的簡化 clock period。
2. R-type 的 logic path 比 `lw` 少多少 ps？
3. 若把 R-type path 改快 100 ps，程式 CPU time 為何不變？用這個 model 解釋。
4. 若只改 faster data-memory 而使 `lw` path 減少 80 ps，所有 instruction 的 clock period 是否都受益？

CH04-Q02 五階時空圖

設計有 IF、ID、EX、MEM、WB 五階，stage 可完全重疊。執行 7 條 instructions，且沒有 hazards。

1. 畫出每條 instruction 的五個 stage。
2. 計算最後一條 instruction 的 WB cycle。
3. 計算完成7條instructions所需cycles與平均IPC。另明定pipeline週期200 ps、未管線化處理器每條需1000 ps，兩者皆忽略額外latch成本，求相同7條指令的時間speedup。
4. 若 stage latency 都是 200 ps，分別寫出單筆 latency 與完成這 7 條的總時間。

Chapter 4 練習 （續）

CH04-Q03 Forwarding

```
I1: add $8, $9, $10
I2: sub $11, $8, $12
I3: and $13, $11, $8
```

假設每階段一 cycle，register read 在 ID、write 在 WB，ALU result 可在 EX/MEM pipeline register 取得。

1. 完成三條 instructions 的 pipeline 時空圖。
2. I2 的 `$8` 由哪一個 pipeline register 前送？
3. I3 的 `$8` 是否可從 MEM/WB 前送？I3 的 `$11` 從哪裡前送？
4. 若應前送卻漏掉，最後 `$13` 可能使用什麼舊值？

CH04-Q04 Load-use

```
I1: lw  $2, 20($1)
I2: and $4, $2, $5
I3: or  $6, $1, $7
```

1. 以 hazard condition 判定 I2 是否需要 stall。
2. 在 ID 偵測 hazard 時，PC、IF/ID 與 ID/EX 各要採取什麼動作？
3. 完成含一個 bubble 的時空圖。
4. 若把 I2 改成 `sw $2, 24($1)`，source semantics 是否讓 hazard 判定不同？說明。

CH04-Q05 Branch prediction

2-bit saturating predictor 狀態為 SN、WN、WT、ST。規則如下：

- taken 時往 ST 方向移動一格。
- not taken 時往 SN 方向移動一格。
- 在 SN 與 ST 不再越界。
- 預測 taken 的狀態是 WT、ST。

分支序列 outcome 為 `T, T, N, T, N, N, N`，initial state 為 WN。

Chapter 4 練習 （續）

1. 列出每次 prediction、outcome 與 state。
2. 計算 misprediction count 與 accuracy。
3. 假設 predictor 同時提供正確 target；方向預測每次錯誤都 flush 2 個 wrong-path instruction slots，方向預測正確則不 flush。估算本序列的 wrong-path instruction slots。
4. 為何只用 branch direction accuracy 不足以預測 pipeline penalty？

CH04-Q06 Precise exception

單發射五階 pipeline 中，instruction `I1` 已在先前 cycle commit。本 cycle 的合法 stage snapshot 是 `I2` 在 WB、`I3` 在 MEM、`I4` 在 EX、`I5` 在 ID、`I6` 在 IF。`I4` 在 EX 發生 arithmetic overflow；假設 `I2`、`I3` 沒有 exception。

1. 哪些 instructions 的 architectural writes 應保留？
2. 哪些 instructions 的 writes 應阻止？
3. Exception buffer 應保存哪些 PC 或 cause 資訊？Handler 最早可在何時取得一致的 architecture state？
4. 若有 out-of-order core，ROB 如何協助保持 precise state？

CH04-Q07 Hazard control logic

令 ID/EX 為 instruction 正在 EX stage 的 control bits，IF/ID 為 waiting in ID 的 consumer。

1. 寫出 load-use stall 的條件式，檢查 `ID/EX.MemRead`、`ID/EX.RegisterRt`、`IF/ID.RegisterRs`、`IF/ID.RegisterRt`。
2. 為何需要確認 producer destination 不是 `$zero`？
3. I2 是不使用 `rt` 的 `lw` 時，不可因 index 相等而一律 stall；可以用哪個 control 條件修正？
4. 若 from EX/MEM 與 MEM/WB 同 cycle 都可前送同一 register，應選哪個 value？說明 oldest/newest 原則。

CH04-Q08 ILP 與多發射

某程式有 10,000 條 instructions。四發射處理器在 60% cycles 能穩定發射 4 條，在 30% cycles 發射 2 條，在 10% cycles 發射 1 條；clock frequency 是 2 GHz。忽略 startup、drain 與其他 stall 的獨立影響。

Chapter 4 練習 （續）

1. 比例的分母是 cycles。先以每cycle發射量的加權平均求IPC，再求完成10000條所需 cycles。
2. 計算執行時間。
3. 若改成雙發射且 every cycle 發射 2 條，但 clock frequency 是 2.4 GHz，哪個較快？
4. 為什麼這個模型不能預測 branch misprediction、cache miss 或 register-port bottleneck 的效果？

CH04-Q09 多週期 FSM、CPI 與時鐘

假設多週期 controller 的 state path 如下：

- R-type：4 cycles。
- load：5 cycles。
- store：4 cycles。
- branch/jump：3 cycles。

Instruction mix 是 45% R-type、20% branch/jump、20% load、15% store；每個 controller cycle 為 300 ps。

1. 由上述路徑計算 weighted CPI。
2. 執行 10^9 instructions 的 CPU time 為何？
3. 若比較基準 single-cycle 的 clock period 是 800 ps、CPI=1，哪個設計在此模型較快？
4. 若要讓多週期設計達到與 single-cycle 相同時間，其他條件不變時，cycle time 最大可為多少？

CH04-Q10 Branch slot、flush 與精確例外

1. 20% instructions 是 branch，其中 60% taken。Predictor 一律預測 not taken，branch 在 EX 解出；no-delay-slot pipeline 每次 taken misprediction flush 2 個 younger instruction slots。求 per-instruction misprediction cycle cost。
2. 同一程式另假設 10% instructions 是 immediate load-use consumer，每次 forwarding 仍需 1 bubble，且沒有其他 stall。求簡化 CPI。
3. 若改用 classic MIPS 一個 architectural delay slot，taken branch 只 flush delay slot 之後的 1 個 wrong-path slot。說明為何不能直接把 penalty 從 2 改成 1 而不重新計算 instruction count。
4. 令 faulting `I4` 在 EX 時，`I2` 在 WB、`I3` 在 MEM、`I5` 在 ID、`I6` 在 IF，且 `I1` 已在更早 cycle commit。列出 exception 捕捉、older drain、younger flush 與 handler redirect 的 cycle actions。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 4 練習 （續）

CH04-Q11 從 machine words 重建三個 mux 與 next-PC

沿用講義的無delay-slot單週期subset，PC=0x00400020、s0=0x1000、s1=7、s2=5、t2=0x12345678，Mem[0x100c]=0xabcdef01，其餘registers初始0。每一小題獨立從此狀態開始；word memory存取對齊，無overflow例外。

1. 解碼0x02324021、0x8e09000c、0xae0afffc。列RegDst、ALUSrc、MemtoReg、WriteReg、ALU inputs/result、memory/register提交效果。
2. 對0x1232fffe，計PC4、BT、Zero與nextPC；再將s1改成5重算。
3. 執行j field0、但PC=0x0fffffc時，求nextPC。為何必須用PC+4的高4 bits？
4. 若sw的ALUSrc誤設0，address會變什麼？若address正確但store data錯接sign-extended immediate，哪個值會被寫入？分別指出出錯的wire，不要只寫「control錯了」。

CH04-Q12 從舊值、load 與後續相依重建時序

使用講義「逐 cycle 數例」的四條指令、初值與五階模型；WB 前半 cycle 寫入，ID 後半 cycle 讀取。

程式如下；初值為 `$1=0x1000`、`$6=0x30`、`$7=0x03`、`$5=0x0f`、`$9=0x80`，`Mem[0x1014]=0x3c`，其餘相關暫存器為 0。無 cache miss、分支或結構衝突，存取對齊且無例外。

```
I1  add $2,$6,$7
I2  lw  $2,20($1)
I3  and $4,$2,$5
I4  or  $8,$4,$9
```

1. 有 EX/MEM、MEM/WB 至 EX 的前送時，列出四條指令的 IF、ID、EX、MEM、WB cycles；明列重複停留的 stage。
2. C5 結束的 ID/EX 可能暫存哪個 `$2` 值？I3 的 EX 為何仍能算對？寫出 C6、C7 的前送來源、數值與最終 `$4`、`$8`。
3. 改為完全沒有 forwarding，重排時序並求總 cycles；不得改變 register-file 時序約定。
4. 若有 forwarding，但將 I3 改為 `and $4,$10,$5`，且 `$10=0x55`，是否還需要 load-use stall？求總 cycles 與 `$8`。

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study